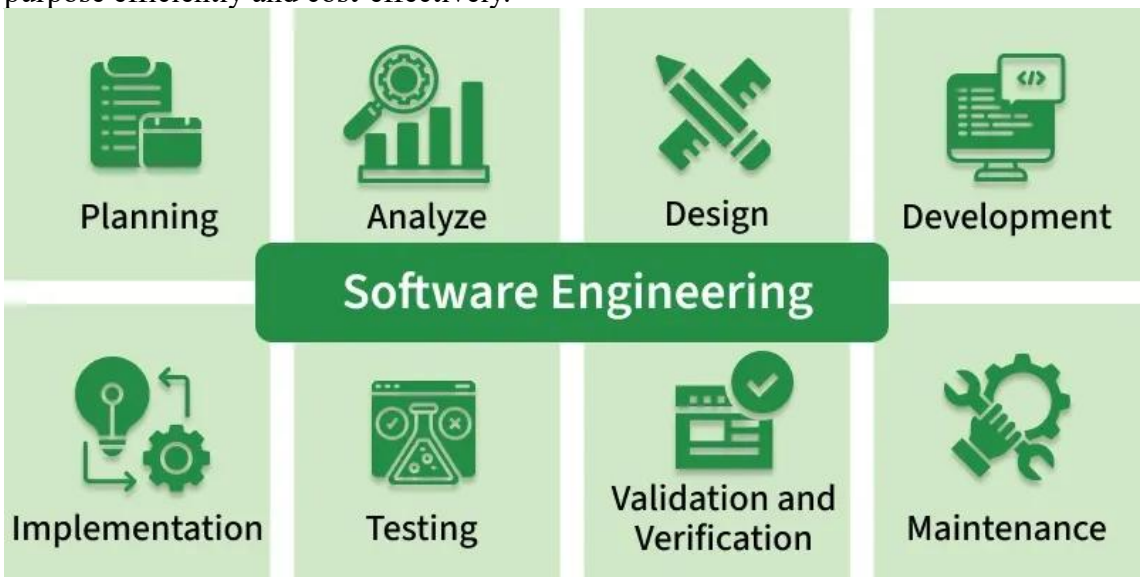
UNIT 1

Topic 1: Introduction to Software Engineering

Software is a program or set of programs containing instructions that provide the desired functionality. Engineering is the process of designing and building something that serves a purpose efficiently and cost-effectively.



Software Engineering

- Software Engineering is the systematic process of designing, developing, testing, and maintaining software using techniques like requirements analysis, design, testing, and maintenance.
- It ensures software is high-quality, reliable, and maintainable, especially for large software systems.
- Software Engineering improves efficiency, quality, and budget management, helping deliver software on time, within budget, and meeting all requirements
- It continuously adopting new tools and methodologies to handle complexity and evolving technologies.

Key Principles

- **Modularity:** Divide software into small, independent parts that are easy to develop and test.
- **Abstraction:** Show only what is necessary and hide internal implementation details.
- **Encapsulation:** Keep data and related functions together and protect them from outside access.
- **Reusability:** Build components that can be reused in different projects to save time and effort.
- **Maintenance:** Regularly update software to fix bugs, improve features, and enhance security.
- **Testing:** Check the software to ensure it works correctly and meets requirements.
- **Design Patterns:** Use proven solutions for common software design problems.
- **Agile Methodologies:** Develop software in small steps with frequent feedback and flexibility.

- **Continuous Integration & Deployment (CI/CD):** Frequently merge code changes and automatically deploy updates.

Main Attributes of Software Engineering

Efficiency:
It provides a measure of the resource requirement of a software product efficiently.
Reliability:
It assures that the product will deliver the same results when used in similar working environment.
Reusability:
This attribute makes sure that the module can be used in multiple applications.
Maintainability:
It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.
Dual Role of Software
There is a dual role of software in the industry. The first one is as a product and the other one is as a vehicle for delivering the product.
1. As a Product

- It delivers computing potential across networks of Hardware.
- It enables the Hardware to deliver the expected functionality.
- It acts as an information transformer because it produces, manages, acquires, modifies, displays, or transmits information.

2. As a Vehicle for Delivering a Product

- It provides system functionality (e.g., payroll system).
- It controls other software (e.g., an operating system).
- It helps build other software (e.g., software tools).

Objectives

1. **Maintainability:** It should be feasible for the software to evolve to meet changing requirements.
2. **Efficiency:** The software should not make wasteful use of computing devices such as memory, processor cycles, etc.
3. **Correctness:** A software product is correct if the different requirements specified in the SRS Document have been correctly implemented.
4. **Reusability:** A software product has good reusability if the different modules of the product can easily be reused to develop new products.
5. **Testability:** Here software facilitates both the establishment of test criteria and the evaluation of the software concerning those criteria.
6. **Reliability:** It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.
7. **Portability:** In this case, the software can be transferred from one computer system or environment to another.
8. **Adaptability:** In this case, the software allows differing system constraints and the user needs to be satisfied by making changes to the software.
9. **Interoperability:** Capability of 2 or more functional units to process data cooperatively.

What Careers Are There in Software Engineering?

A degree in software engineering and relevant experience can be utilized to explore several computing job choices. Software engineers have the opportunity to seek well-paying careers

and professional progress, although their exact possibilities may vary depending on their particular school, industry, and region.
Following are the job choices in software engineering:

- SWE (Software Engineer)
- SDE (Software Development Engineer)
- Web Developer
- Quality Assurance Engineer
- Web Designer
- Software Test Engineer
- Cloud Engineer ·
- Front-End Developer
- Back-End Developer
- DevOps Engineer.
- Security Engineer.

What Tasks do Software Engineers do?

The main responsibility of a software engineer is to develop useful computer programs and applications. Working in teams, you would complete various projects and develop solutions to satisfy certain customer or corporate demands.
Some of the key responsibilities of software engineer are:

- **Requirement Analysis**: Collaborating with stakeholders to understand and gather the requirements to design and develop software solutions.
- **Design and Development**: Creating well-structured, maintainable code that meets the functional requirements and adheres to software design principles.
- **Testing and Debugging**: Writing and conducting unit tests, integration tests, and debugging code to ensure software is reliable and bug-free.
- **Code Review**: Participating in code reviews to improve code quality, ensure adherence to standards, and facilitate knowledge sharing among team members.
- **Maintenance**: Updating and maintaining existing software systems, fixing bugs, and improving performance or adding new features.
- **Documentation**: Writing clear documentation, including code comments, API documentation, and design documents to help other engineers and future developers understand the system.

What Skills do Software Engineers Need?

Following are some must have technical skills to become Software Engineers:

- Coding and computer programming
- Software testing
- Object-oriented design (OOD)
- Software development
-

Changing Nature of Software :

The nature of software has changed a lot over the years.
1.System software: Infrastructure software come under this category like compilers, operating systems, editors, drivers, etc. Basically system software is a collection of programs to provide service to other programs.
2. Real time software: These software are used to monitor, control and analyze real world events as they occur. An example may be software required for weather forecasting. Such software will gather and process the status of temperature, humidity and other environmental parameters to forcast the weather.

3. Embedded software: This type of software is placed in “Read-Only- Memory (ROM)”of the product and control the various functions of the product. The product could be an aircraft, automobile, security system, signalling system, control unit of power plants, etc. The embedded software handles hardware components and is also termed as intelligent software .
4. Business software : This is the largest application area. The software designed to process business applications is called business software. Business software could be payroll, file monitoring system, employee management, account management. It may also be a data warehousing tool which helps us to take decisions based on available data. Management information system, enterprise resource planning (ERP) and such other software are popular examples of business software.
5. Personal computer software :The software used in personal computers are covered in this category. Examples are word processors, computer graphics, multimedia and animating tools, database management, computer games etc. This is a very upcoming area and many big organisations are concentrating their effort here due to large customer base.
6. Artificial intelligence software: Artificial Intelligence software makes use of non numerical algorithms to solve complex problems that are not amenable to computation or straight forward analysis. Examples are expert systems, artificial neural network,signal processing software etc.
7. Web based software: The software related to web applications come under this category. Examples are CGI, HTML, Java, Perl, DHTML etc.

Software Quality Attributes

These attributes can be defined as follows:

- Functionality. The capability to provide functions which meet stated and implied needs when the software is used.
- Reliability. The capability to provide failure-free service.
- Usability. The capability to be understood, learned, and used.
- Efficiency. The capability to provide appropriate performance relative to the amount of resources used.
- Maintainability. The capability to be modified for purposes of making corrections, improvements, or adaptation.
- Portability. The capability to be adapted for different specified environments without applying actions or means other than those provided for this purpose in the product.

Topic 2: Principles of Software Engineering

Seven principles have been determined which form a reasonably independent and complete set. These are: (1) Manage using a phased life-cycle plan. (2) Perform continuous validation. (3) Maintain disciplined product control. (4) Use modern programming practices. (5) Maintain clear accountability for results. (6) Use better and fewer people. (7) Maintain a commitment to improve the process.

Software Engineering principles are a set of **guidelines and best practices** that help developers **design, develop, test, and maintain high-quality software**. These principles ensure that software systems are **reliable, maintainable, efficient, and cost-effective**.

Fundamental Principles of Software Engineering

1. Understand the Problem Clearly

- Requirements must be clearly understood before development begins.
- Continuous interaction with stakeholders is essential.
- Poor understanding leads to rework and project failure.

2. Plan Before You Build

- Proper planning reduces risks and cost.
- Includes project scheduling, resource allocation, and risk analysis.
- "Think before you code" is a key principle.

3. Divide and Conquer (Modularity)

- Break the system into **small, manageable modules**.
- Each module should perform a single function.
- Improves maintainability and readability.

4. Abstraction

- Focus on **what the system does**, not how it does it.
- Hides unnecessary implementation details.
- Helps manage complexity.

5. Information Hiding

- Internal details of a module should not be visible to others.
- Reduces dependency between modules.
- Enhances security and flexibility.

6. Low Coupling and High Cohesion

- **Low coupling**: Modules should have minimal dependency.
- **High cohesion**: Each module should perform one well-defined task.
- Improves maintainability and reusability.

7. Anticipate Change

- Software requirements change over time.
- Design should be flexible and adaptable.
- Use of modular and layered architectures helps.

8. Iterative Development

- Develop software in **small increments**.
- Allows early feedback and error detection.

- Improves quality and reduces risk.

9. Simplicity (KISS Principle)
- *Keep It Simple, Stupid*.
- Avoid unnecessary complexity.
- Simple designs are easier to understand and maintain.

10. Reusability
- Reuse existing components wherever possible.
- Saves time, cost, and effort.
- Encourages use of libraries and frameworks.

11. Consistency
- Use consistent naming, coding styles, and design patterns.
- Improves readability and teamwork.

12. Design for Testability
- Software should be easy to test.
- Early testing helps detect defects sooner.
- Test cases should be planned early.

13. Quality Assurance
- Quality must be built into the process, not inspected at the end.
- Includes reviews, testing, and validation.

14. Documentation
- Proper documentation supports maintenance and future development.
- Should be clear, concise, and updated regularly.

15. User-Centered Design
- Focus on user needs and usability.
- Software should be easy to use and understand.

Topic 3: Software Development Life Cycle (SDLC)

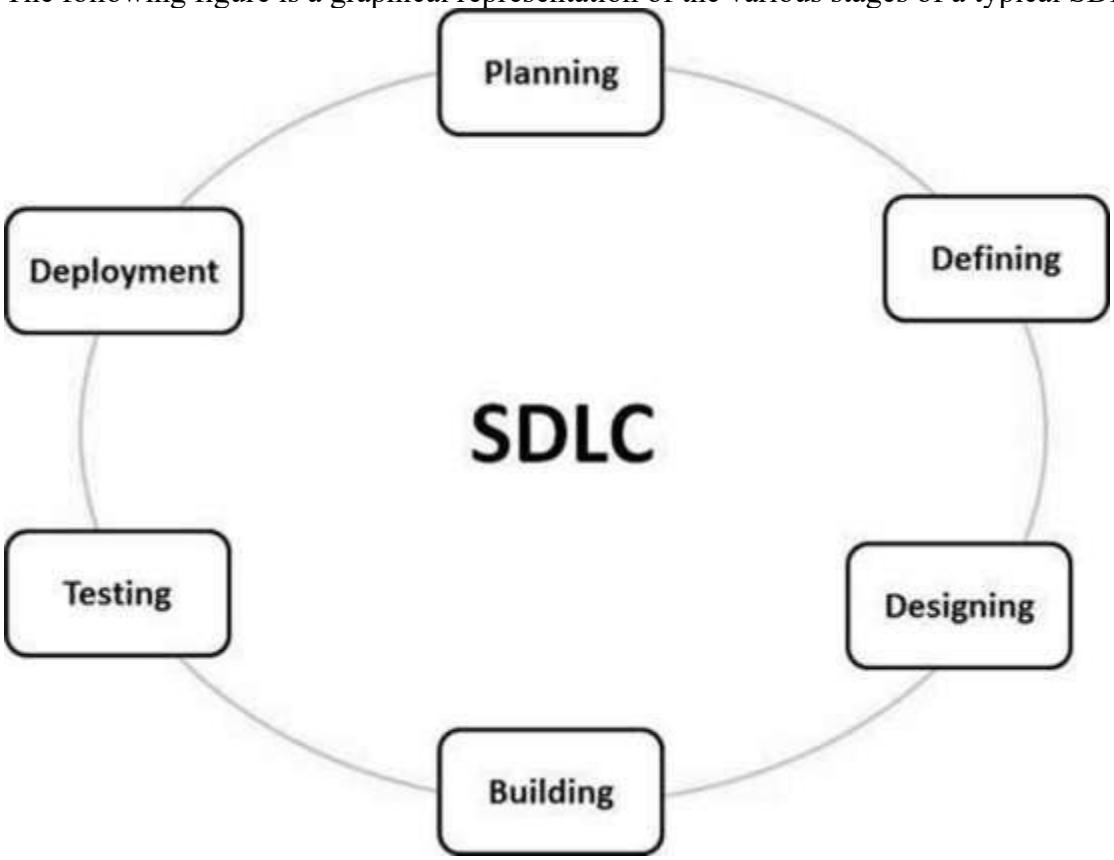
Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality softwares. The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.

- SDLC is the acronym of Software Development Life Cycle.
- It is also called as Software Development Process.
- SDLC is a framework defining tasks performed at each step in the software development process.
- ISO/IEC 12207 is an international standard for software life-cycle processes. It aims to be the standard that defines all the tasks required for developing and maintaining software.

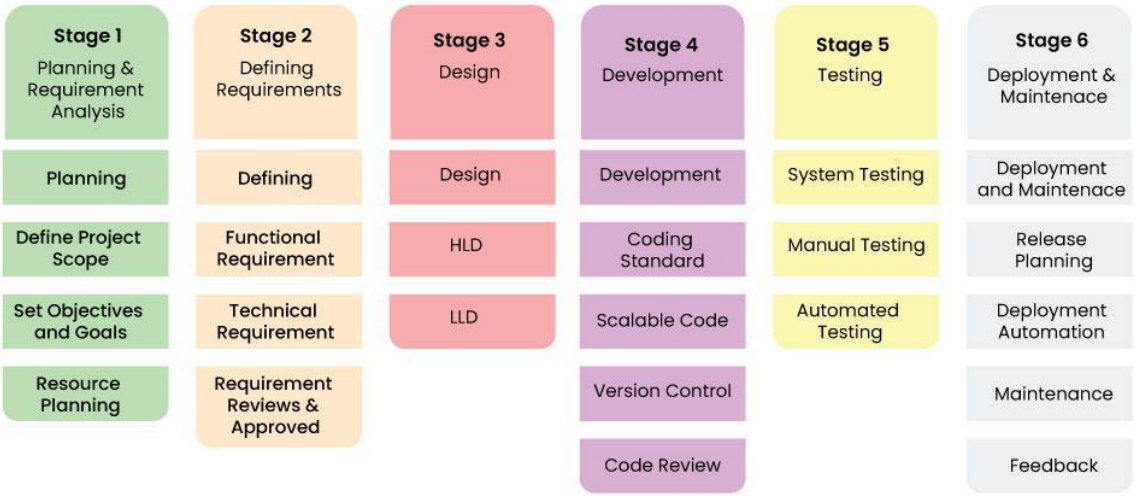
What is SDLC?

SDLC is a process followed for a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

The following figure is a graphical representation of the various stages of a typical SDLC.



SDLC is a collection of these six stages, and the stages of SDLC are as follows:



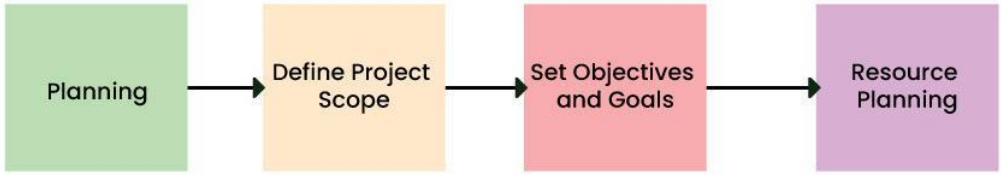
Software Development Life Cycle Model SDLC Stages

The SDLC Model involves six phases or stages while developing any software.

Stage 1: Planning and Requirement Analysis

This is the most fundamental stage. Before writing code, the team must define what they are building and why.

- **Activities:** Feasibility studies (technical, operational, economic), resource allocation, project scheduling, and cost estimation.
- **Output:** Project Plan, Feasibility Report.
- **Key Players:** Senior Engineers, Project Managers, Stakeholders.



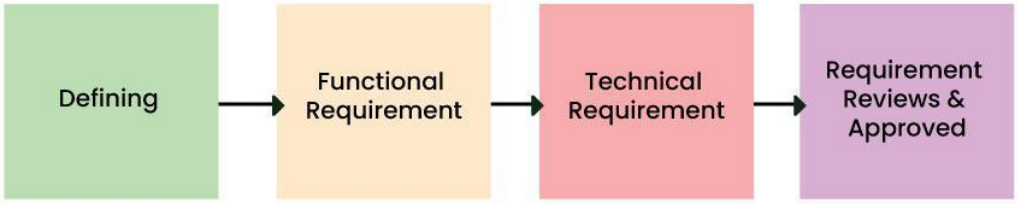
Stage-1 : Planning and Requirement Analysis

Stage 2: Defining Requirements (SRS)

Once the plan is approved, the specific product requirements must be defined and documented. This is done through the **Software Requirement Specification (SRS)** document.

- **Activities:** Gathering detailed functional and non-functional requirements from customers.
- **Output:** SRS Document (The "Bible" for the development team).
- **Key Players:** Business Analysts (BAs), Product Owners

Stage-2: Defining Requirements



Stage-2 : Defining Requirements

Stage 3: Designing Architecture

The requirements are turned into a technical blueprint. This phase defines the overall system architecture and technology stack.

- **High-Level Design (HLD):** Defines the architecture, database design, and relationships between modules.
- **Low-Level Design (LLD):** Defines the logic of individual components, API interfaces, and database tables.
- **Output:** Design Document Specification (DDS).
- **Key Players:** System Architects, Lead Developers.

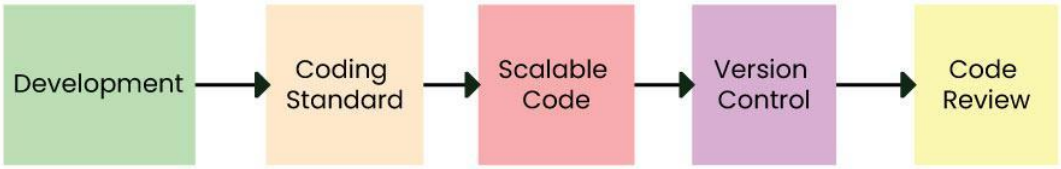
Stage-3: Designing Architecture

Stage 3: Design

Stage 4: Development (Coding)

This is the longest phase where the actual building takes place. Developers write code based on the Design Document.

- **Activities:** Coding, Code Reviews, Unit Testing, and Static Code Analysis.
- **Tools:** Compilers, Debuggers, IDEs (VS Code, IntelliJ), Version Control (Git).
- **Output:** Source Code, Executable Software.
- **Key Players:** Developers (Frontend, Backend, Full Stack).



Stage 4: Development

Stage 5: Testing

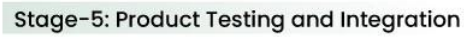
Once the code is written, it moves to the Quality Assurance (QA) team. The goal is to find bugs before the customer does.

Types of Testing:

- **Unit Testing:** Testing individual functions.
- **Integration Testing:** Ensuring modules work together.
- **System Testing:** Testing the entire application flow.
- **User Acceptance Testing (UAT):** Verifying the software meets business needs.

Output: Bug Reports, Test Cases, Quality Report.

Key Players: QA Engineers, Testers.

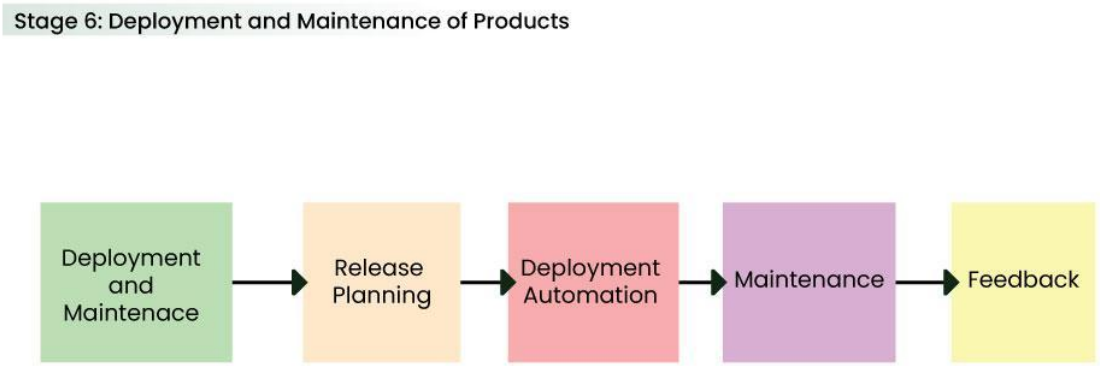


Stage 5: Testing

Stage 6: Deployment

The software is released to the end-users. In modern DevOps environments, this is often automated via CI/CD pipelines.

- **Activities:** Setting up production environments, deploying code, and smoke testing the live environment.
- **Output:** Live Application.
- **Key Players:** DevOps Engineers, Release Managers.



Stage 6: Deployment and Maintenance

Stage 7: Maintenance

The cycle doesn't end at deployment. Software must be maintained to remain useful.

- **Activities:** Bug fixing, upgrading libraries, performance tuning, and adding new features.
- **Output:** Patches, Updates, New Versions.
- **Key Players:** Support Engineers, Developers.

Software Development Life Cycle Models

Software Development Models are structured frameworks that guide the planning, execution, and delivery of software projects. They define the **sequence of development stages**, such as requirements, design, coding, testing, and deployment.

Common Models

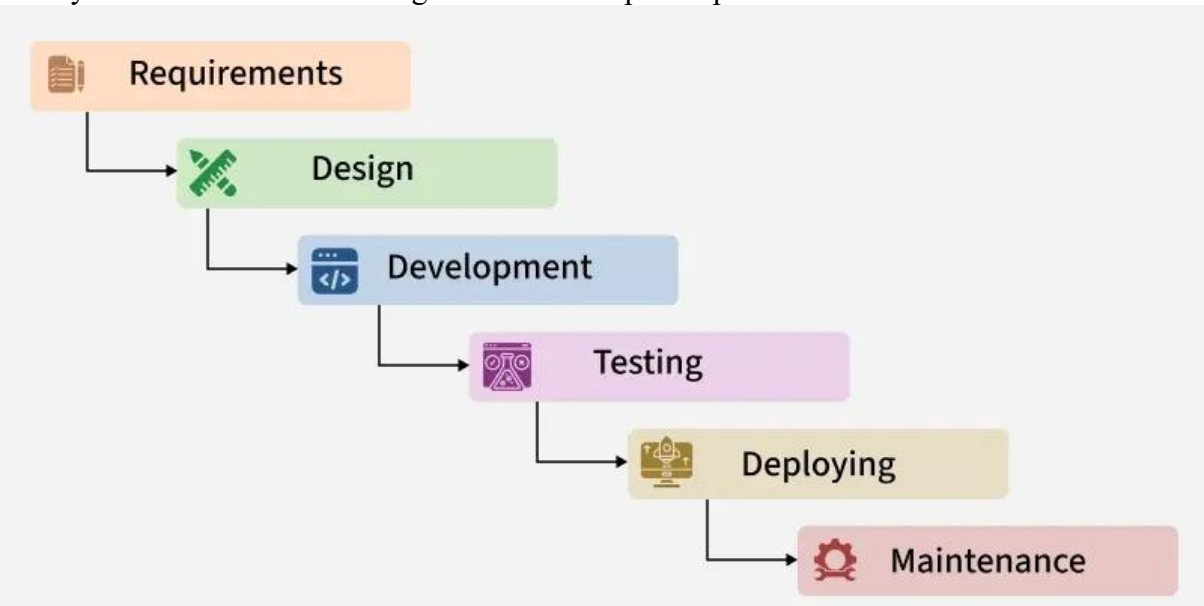
- Waterfall Model
- Agile Model
- V-Model
- Spiral Model
- Incremental Model
- RAD Model

What is the need for SDLC?

SDLC is a method, approach, or process that is followed by a software development organization while developing any software. SDLC models were introduced to follow a disciplined and systematic method while designing software. With the software development life cycle, the process of software design is divided into small parts, which makes the problem more understandable and easier to solve. SDLC comprises a detailed description or step-by-step plan for designing, developing, testing, and maintaining the software.

Topic 4: Waterfall Model - Software Engineering

The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next. It does not allow backtracking and permits only minimal changes once a phase is completed. The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.



Features of the Waterfall Model

- **Sequential Approach**
  Development follows a linear, step-by-step process where each phase is completed before moving to the next.
- **Document-Driven Process**
  Detailed documentation is created at every stage to clearly define requirements, design, and progress.
- **Quality Control**
  Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.
- **Detailed Planning**
  The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.

Phases of Waterfall Model

Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are follow:

1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:**
  Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.
- **Requirement Specification:**
  The approved requirements are documented in the **Software Requirement Specification (SRS)**, which serves as a formal agreement between the customer and the development team.

2. Design

In this phase, the requirements from the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**
  Defines the overall system architecture, major components, and their interactions.
- **Low-Level Design (LLD):**
  Provides detailed design of each component, including logic and data flow, to guide developers during coding.
- All designs are documented in the **Software Design Document (SDD)**.

3. Development

This phase involves converting the design into actual working software.

- Developers write source code based on the design documents.
- Suitable programming languages and tools are used.
- **Unit testing** is performed to verify that each module works correctly on its own.

4. Testing and Deployment

This phase ensures that the integrated software functions correctly and is successfully delivered for real world use.

Testing: After unit testing, software modules are integrated incrementally and tested at each stage. Once all modules are successfully integrated, full system testing is performed to ensure the system functions as expected.

- **Alpha Testing:** Performed by the development team.
- **Beta Testing:** Performed by selected end users.
- **Acceptance Testing:** Performed by the customer to approve the software.

Deployment: After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

5. Maintenance

Maintenance ensures the software continues to function effectively after deployment.

- **Corrective Maintenance:** Fixes errors found after release.
- **Perfective Maintenance:** Enhances features based on user needs.
- **Adaptive Maintenance:** Updates software to work in new environments, such as new hardware or operating systems.

Advantages

- **Easy to Understand:**
  The model is straightforward and simple, making it easy to learn and follow.
- **Sequential Processing:**
  Each phase is completed one at a time, ensuring an organized development flow.
- **Well-Defined Stages:**
  Every phase has clear objectives and deliverables, reducing confusion.
- **Clear Milestones:**
  Progress is easy to track due to clearly defined milestones.
- **Strong Documentation:**
  All processes, actions, and results are thoroughly documented.
- **Encourages Best Practices:**
  Promotes disciplined practices such as define before design and design before c*oding*.
- **Suitable for Small and Stable Projects:**
  Works well for smaller projects where requirements are clearly understood and unlikely to change.

When to Use Waterfall Model?

- **Well-Defined Requirements:**
  Requirements are clear, stable, and fully documented before development begins.

- **Minimal Changes Expected:**
  The project scope is unlikely to change during development.
- **Small to Medium-Sized Projects:**
  Suitable for projects with limited complexity and a clear development path.
- **Predictable and Low Risk:**
  Risks are known, manageable, and can be addressed early in the development cycle.
- **Strict Regulatory Compliance:**
  Ideal when detailed documentation and adherence to standards or regulations are mandatory.
- **Client Prefers a Sequential Approach:**
  Best when the client wants a step-by-step, linear development process.
- **Limited Resources:**
  Works well when resources are limited and need careful planning and allocation.

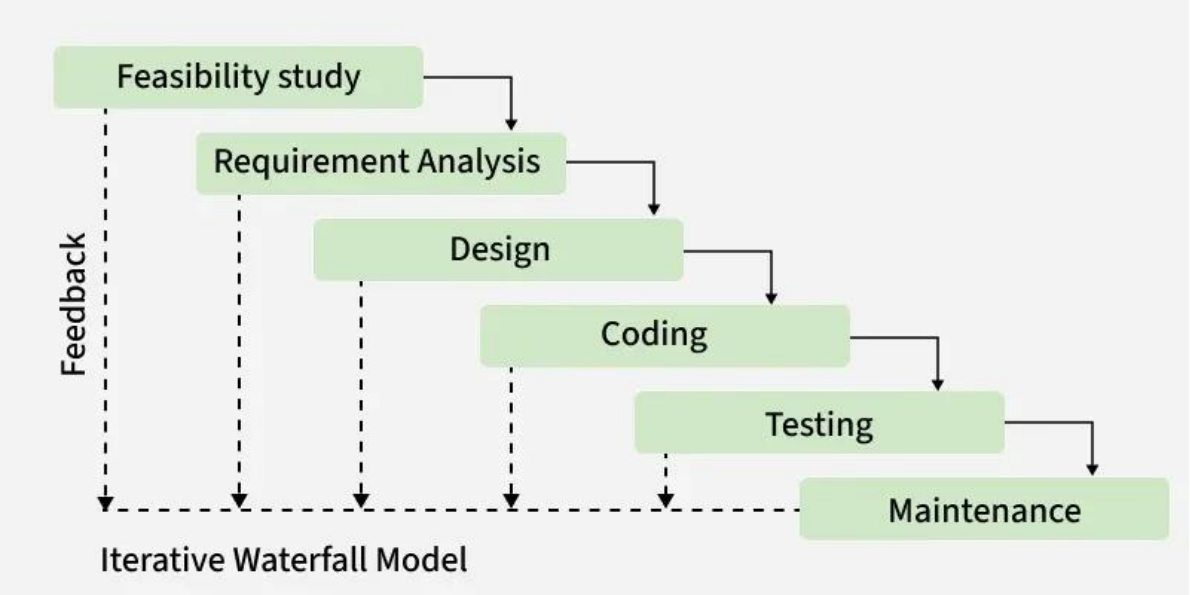
Example of Waterfall Model

Below is a real-world example of the Waterfall Model using an **Online Food Delivery System**.

Topic 5: Iterative Waterfall Model - Software Engineering

The Classical Waterfall Model is difficult to use in real-world projects because it does not allow changes once a phase is completed. To overcome this limitation, the Iterative Waterfall Model was introduced.

It follows the same phase-by-phase structure as the Waterfall Model but adds feedback paths, allowing earlier phases to be revisited and improved.



Iterative Waterfall Model

The Iterative Waterfall Model is a software development approach that combines:

- The sequential structure of the Waterfall Model
- The flexibility of iteration and feedback

Each phase can send feedback to its previous phase, making it possible to correct mistakes early instead of waiting until the end.

1. Sequential development with feedback loops
2. Errors are corrected in the same phase where they occur
3. Changes are reflected in later phases
4. Improves efficiency compared to the classical Waterfall Model
5. Reduces time and cost of fixing errors

Example:

Developing a mobile banking app where requirements are defined first, the app is designed, developed, tested, and deployed. After feedback, errors are fixed and features are improved by revisiting earlier phases. This cycle is repeated until the final, stable version is achieved.

Phases of Iterative Waterfall Model

1. Requirements Gathering

- Business owners and developers discuss project goals
- Functional and non-functional requirements are collected
- Requirements are clearly documented

2. Design

- A preliminary system design is created
- Architecture, database, and interface design are planned
- Design can be revised based on feedback

3. Implementation

- Developers write the code based on the design
- Modules are developed step by step

- Errors found can send feedback to the design phase

4. Testing

- The software is tested to ensure it meets requirements
- Bugs and issues are identified early
- Necessary corrections are made through feedback loops

5. Deployment

- The software is deployed and made available to users
- Final checks are performed

6. Review and Improvement

- System performance is reviewed
- Improvements and refinements are made
- The process repeats until the product meets objectives

When to use Iterative Waterfall Model?

- Requirements are mostly clear but may need refinement
- Team is learning new technologies
- Project has a high risk of future changes
- Large projects with limited resources
- When early error detection is important

Application

- Projects with stable core requirements
- Systems using new or unfamiliar technology
- Large projects divided into manageable phases
- High-risk projects requiring regular validation

Why is iterative waterfall model used?

- Allows feedback between phases
- Helps detect and fix errors early
- Reduces rework and development cost
- Keeps the project aligned with goals

Advantages

- Early detection and correction of errors
- Improved collaboration between developers and clients
- Better flexibility than classical Waterfall
- Regular testing improves quality
- Faster delivery of refined versions
- Reduced project risk

Drawbacks

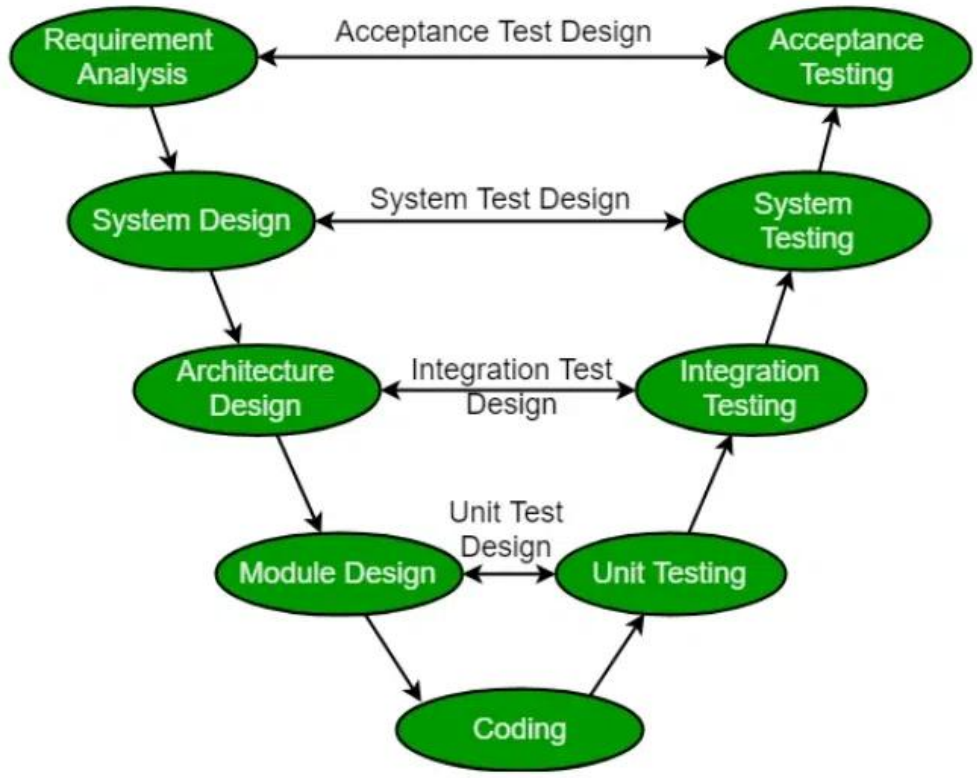
- Changes after development starts are difficult
- No partial or incremental delivery
- Phases cannot overlap
- No formal risk management mechanism
- Limited customer interaction after requirements phase

Topic 7: SDLC V-Model - Software Engineering

The SDLC V-Model is a type of Software Development Life Cycle (SDLC). It is a method that includes testing and validation alongside each development phase. It creates a structure like the letter 'V,' which includes various phases that we will discuss in detail.

Phases of SDLC V-Model

The V-Model, which includes the Verification and Validation it is a structural approach to software development. The following are the different Phases of the V-Model of the SDLC.



SDLC V-Model

1. V-Model Verification Phases

This is where the process begins. The first step is to gather and understand the customer's needs for the software. The goal is to define the scope of the project clearly to make sure everyone is on the same page.

It involves a static analysis technique (review) done without executing code. It is the process of evaluation of the product development phase to find whether specified requirements are met.

There are several Verification phases in the V-Model:

1. Business Requirement Analysis

This is the first step of the designation of the development cycle where product requirement needs to be cured from the customer's perspective. These phases include proper communication with the customer to understand their requirements.

These are the very important activities that need to be handled properly, as most of the time customers do not know exactly what they want, and they are not sure about it at that time then we use an **acceptance test design** planning which is done at the time of business requirement it will be used as an **input** for acceptance testing.

2. System Design
In this phase, the overall structure of the software is planned out. The team develops both the high-level design (how the system will be structured) and detailed design (how the individual components will work).
Design of the system will start when the overall we are clear with the product requirements, and then need to design the system completely. This understanding will be at the beginning of complete under the product development process. these will be beneficial for the future execution of test cases.
3. Architectural Design
In this stage, architectural specifications are comprehended and designed. Usually, several technical approaches are put out, and the ultimate choice is made after considering both the technical and financial viability. The system architecture is further divided into modules that each handle a distinct function. Another name for this is High-Level Design (HLD).
At this point, the exchange of data and communication between the internal modules and external systems are well understood and defined. During this phase, integration tests can be created and documented using the information provided.
4. Module Design
This phase, known as Low-Level Design (LLD), specifies the comprehensive internal design for every system module. Compatibility between the design and other external systems as well as other modules in the system architecture is crucial. Unit tests are a crucial component of any development process since they assist in identifying and eradicating the majority of mistakes and flaws at an early stage. Based on the internal module designs, these unit tests may now be created.
5. Coding Phase
This is where the software is actually built. Developers write the code based on the design created in the previous phase.
The Coding step involves writing the code for the system modules that were created during the Design phase. The system and architectural requirements are used to determine which programming language is most appropriate.
The coding standards and principles are followed when performing the coding. Before the final build is checked into the repository, the code undergoes many code reviews and is optimized for optimal performance.
2. V-Model Validation Phases
It involves dynamic analysis techniques (functional, and non-functional), and testing done by executing code. Validation is the process of evaluating the software after the completion of the development phase to determine whether the software meets the customer's expectations and requirements.
1. Unit Testing
In **Unit testing**, unit Test Plans are developed during the module design phase. These Unit Test Plans are executed to eliminate bugs in code or unit level.
2. Integration testing
After completion of unit testing **Integration testing** is performed. In integration testing, the modules are integrated and the system is tested. Integration testing is performed in the Architecture design phase. This test verifies the communication of modules among themselves.
3. System Testing
System testing tests the complete application with its functionality, inter-dependency, and communication. It tests the functional and non-functional requirements of the developed application.

4. User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets the user's requirement and the system is ready for use in the real world.

Industrial Challenge of V-Model

As the industry has evolved, the technologies have become more complex, increasingly faster, and forever changing, however, there remains a set of basic principles and concepts that are as applicable today as when IT was in its infancy.

- Accurately define and refine user requirements.
- Design and build an application according to the authorized user requirements.
- Validate that the application they had built adhered to the authorized business requirements.

Importance of V-Model

The V-Model is an important part of the SDLC, and the process is structured and sequential throughout all the testing. Here is why the V-Model is important:

1. Early Defect Identification

By incorporating verification and validation tasks into every stage of the development process, the V-Model encourages early testing. This lowers the cost and effort needed to remedy problems later in the development lifecycle by assisting in the early detection and resolution of faults.

2. Determining the Phases of Development and Testing

The V-Model contains a testing phase that corresponds to each stage of the development process. By ensuring that testing and development processes are clearly mapped out, this clear mapping promotes a methodical and orderly approach to software engineering.

3. Prevents "Big Bang" Testing

Testing is frequently done at the very end of the development lifecycle in traditional development models, which results in a "Big Bang" approach where all testing operations are focused at once. By integrating testing activities into the development process and encouraging a more progressive and regulated testing approach, the V-Model prevents this.

4. Improves Cooperation

At every level, the V-Model promotes cooperation between the testing and development teams. Through this collaboration, project requirements, design choices, and testing methodologies are better understood, which improves the effectiveness and efficiency of the development process.

5. Improved Quality Assurance

Overall quality assurance is enhanced by the V-Model, which incorporates testing operations at every level. Before the program reaches the final deployment stage, it makes sure that it satisfies the requirements and goes through a strict validation and verification process.

Principles of V-Model

- **Large to Small:** In V-Model, testing is done in a hierarchical perspective, for example, requirements identified by the project team, creating High-Level Design, and Detailed Design phases of the project. As each of these phases is completed the requirements, they are defining become more and more refined and detailed.
- **Data/Process Integrity:** This principle states that the successful design of any project requires the incorporation and cohesion of both data and processes. Process elements must be identified at every requirement.
- **Scalability:** This principle states that the V-Model concept has the flexibility to accommodate any IT project irrespective of its size, complexity, or duration.
- **Cross Referencing:** A direct correlation between requirements and corresponding testing activity is known as cross-referencing.

Tangible Documentation:
This principle states that every project needs to create a document. This documentation is required and applied by both the project development team and the support team. Documentation is used to maintain the application once it is available in a production environment.

Why preferred?

- It is easy to manage due to the rigidity of the model. Each phase of V-Model has specific deliverables and a review process.
- Proactive defect tracking – that is defects are found at an early stage.

When to Use of V-Model?

- **Traceability of Requirements:** The V-Model proves beneficial in situations when it's imperative to create precise traceability between the requirements and their related test cases.
- **Complex Projects:** The V-Model offers a methodical way to manage testing activities and reduce risks related to integration and interface problems for projects with a high level of complexity and interdependencies among system components.
- **Waterfall-Like Projects**: Since the V-Model offers an approachable structure for organizing, carrying out, and monitoring testing activities at every level of development, it is appropriate for projects that use a sequential approach to development, much like the waterfall model.
- **Safety-Critical Systems:** These systems are used in the aerospace, automotive, and healthcare industries. They place a strong emphasis on rigid verification and validation procedures, which help to guarantee that essential system requirements are fulfilled and that possible risks are found and eliminated early in the development process.

Advantages of V-Model

- This is a highly disciplined model and Phases are completed one at a time.
- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- This model focuses on verification and validation activities early in the life cycle thereby enhancing the probability of building an error-free and good quality product.
- It enables project management to track progress accurately.
- The V-Model provides a clear and structured process for software development, making it easier to understand and follow.
- The V-Model places a strong emphasis on testing, which helps to ensure the quality and reliability of the software.
- The V-Model provides a clear link between the requirements and the final product, making it easier to trace and manage changes to the software.
- The clear structure of the V-Model helps to improve communication between the customer and the development team.

Disadvantages of the V-Model

- The V-Model is a linear and sequential model, which can make it challenging to adapt to changing requirements or unforeseen events.
- The V-Model can be time-consuming, as it requires a lot of documentation and testing.
- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.
- This model does not support iteration of phases.

- The V-Model places a strong emphasis on documentation, which can lead to an overreliance on documentation at the expense of actual development work.

S.No	Verification	Validation
1.	Addresses whether "Are you building a software right?"	Addresses whether "Are you building a right software?"
2.	Ensures that the software system meets all requirements.	Ensures that the functionalities meet the intended behaviour.
3.	It takes place first including checking for deocumentation, code, etc.	It occurs after Verification and involves checking of the overall products.
4.	Done by Developers	Done by Testers.
5.	It has static activities like, collecting reviews, walk-throughs etc.	It has dynamic activities like, executing the software against requirements.

Topic 8: Incremental Process Model

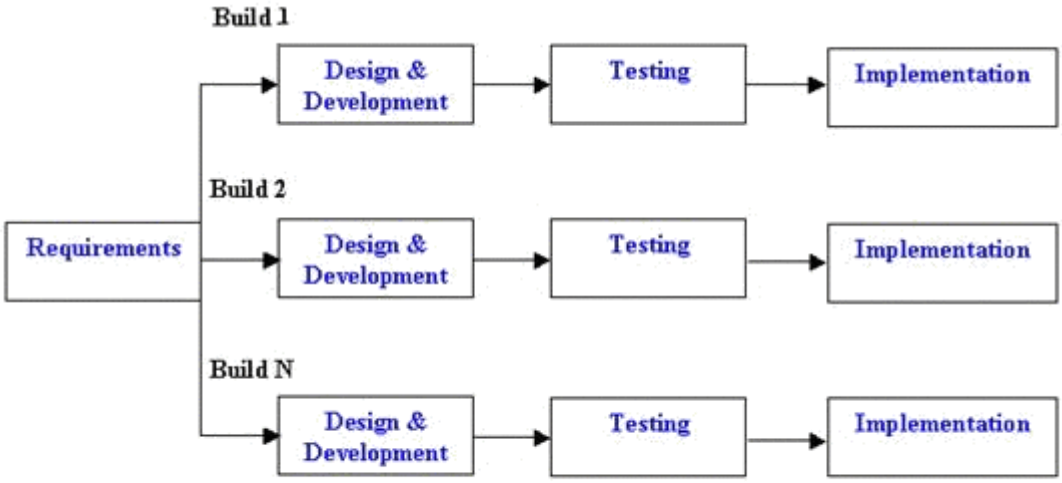
Definition

The **Incremental Process Model** is a software development model in which the system is designed, implemented, and tested **incrementally**.

Instead of delivering the entire product at once, the software is developed and delivered in **small functional parts (increments)**. Each increment adds new features to the previously released version.

Basic Idea

- The **overall requirements** are divided into multiple **modules/increments**.
- Each increment goes through all phases of the **SDLC**:
  - Requirements analysis
  - Design
  - Implementation
  - Testing
- After completing an increment, a **working version** of the software is delivered.
- New functionality is added with every subsequent increment until the full system is complete.



Incremental Life Cycle Model

Phases of Incremental Model

Each increment follows these phases:

1. Requirement Analysis

- Requirements are identified and prioritized.
- Only requirements for the **current increment** are analyzed in detail.
- High-priority features are developed first.

2. Design

- System architecture is designed at a high level initially.
- Detailed design is done **increment-wise**.
- Ensures compatibility with future increments.

3. Implementation (Coding)

- Code is written for the selected features of the increment.
- Each increment is a **complete, executable subset** of the system.

4. Testing

- Unit testing, integration testing, and system testing are performed.

- Ensures the increment works correctly and integrates with previous increments.

5. Deployment & Review

- Increment is delivered to the customer.
- Customer feedback is collected and used for future increments.

Features of Incremental Model

- Software is developed in **multiple releases**
- **Early delivery** of useful software
- **Customer feedback** after every increment
- Requirements can evolve over time
- Testing and debugging are easier due to smaller modules

Advantages

1. **Early Delivery of Software**
   - Working software is available early in the lifecycle.
2. **Reduced Risk**
   - High-risk features are implemented first.
3. **Customer Feedback**
   - Continuous user involvement improves quality.
4. **Flexibility**
   - Changes in requirements can be easily accommodated.
5. **Easier Testing & Debugging**
   - Smaller increments are easier to test and maintain.
6. **Better Resource Utilization**
   - Development teams can work incrementally and efficiently.

Disadvantages

1. **Requires Good Planning**
   - Overall architecture must be well-designed from the beginning.
2. **Integration Issues**
   - Poor design may cause difficulties when integrating increments.
3. **Higher Management Overhead**
   - Multiple releases require strong coordination.
4. **Not Suitable for Small Projects**
   - Overhead may outweigh benefits.

When to Use Incremental Model

- When **requirements are well understood** but can evolve
- When **early delivery** is important
- Large systems divided into **independent modules**
- Projects with **limited time** but flexible scope
- Customer is available for **continuous feedback**

Example

Consider an **online banking system**:

- Increment 1: User login, account viewing
- Increment 2: Fund transfer
- Increment 3: Bill payments, reports

Each increment is usable and adds value.

Topic 9: Evolutionary Process Model

Definition

The **Evolutionary Process Model** is a software development approach in which the system is **developed in iterations**, and the software **evolves progressively** based on **customer feedback and changing requirements**.

Instead of defining all requirements upfront, the product is refined through **repeated cycles of development, evaluation, and enhancement**.

Basic Concept

- Requirements are **not fully known at the beginning**.
- A **basic version** of the software is built first.
- The software is **continuously refined** through multiple iterations.
- Each iteration delivers a **more complete and improved version** of the system.
- Customer involvement is continuous.

Types of Evolutionary Process Models

1. Prototyping Model

2. Spiral Model

Both follow evolutionary principles but differ in focus.

1. Prototyping Model

Definition

The **Prototyping Model** builds a **working prototype** of the system to understand requirements clearly. The prototype is refined based on user feedback until the final system is developed.

Steps in Prototyping Model

1. Requirement Gathering (Initial)
2. Quick Design
3. Build Prototype
4. User Evaluation
5. Refinement of Prototype
6. Final Product Development

Types of Prototypes

- **Throwaway (Rapid) Prototype** – discarded after requirement clarification
- **Evolutionary Prototype** – gradually converted into the final system

2. Spiral Model

Definition

The **Spiral Model** is a **risk-driven evolutionary model** that combines **iterative development** with **systematic risk analysis**.

Phases of Spiral Model (Each Loop)

1. Planning
2. Risk Analysis
3. Engineering
4. Evaluation

Each loop of the spiral represents a **development iteration**.

General Phases of Evolutionary Model

1. Initial Requirement Analysis

- Only **high-level requirements** are gathered.

- Detailed requirements evolve later.

2. Design and Development
- A working version is built.
- Focus is on core functionality.

3. User Evaluation
- Customer reviews the current version.
- Feedback is collected.

4. Refinement
- Improvements and changes are incorporated.
- System evolves in functionality and quality.

5. Final System
- After several iterations, a stable and complete system is delivered.

Characteristics of Evolutionary Model
- Iterative and incremental in nature
- Continuous user involvement
- Flexible requirements
- Early delivery of partial system
- Emphasis on feedback and refinement

Advantages
1. **Handles Changing Requirements**
   - Ideal when requirements are unclear initially.
2. **Customer Satisfaction**
   - Frequent feedback ensures correct product development.
3. **Early Working Software**
   - Partial but usable system available early.
4. **Reduced Risk**
   - Issues are identified early in iterations.
5. **Better Quality**
   - Continuous testing and refinement improve quality.

Disadvantages
1. **Difficult to Manage**
   - Requires strong planning and control.
2. **Scope Creep**
   - Continuous changes may delay completion.
3. **Higher Cost**
   - Multiple iterations increase cost and effort.
4. **Not Suitable for Small Projects**
   - Overhead may be unnecessary.

When to Use Evolutionary Model
- Requirements are **uncertain or changing**
- User interaction is essential
- Complex and large systems
- New technology is involved
- High-risk projects

Example

Hospital Management System
- Version 1: Patient registration

- Version 2: Doctor scheduling
- Version 3: Billing and reports

Each version improves based on feedback.

Topic 10: Prototyping Model

Definition

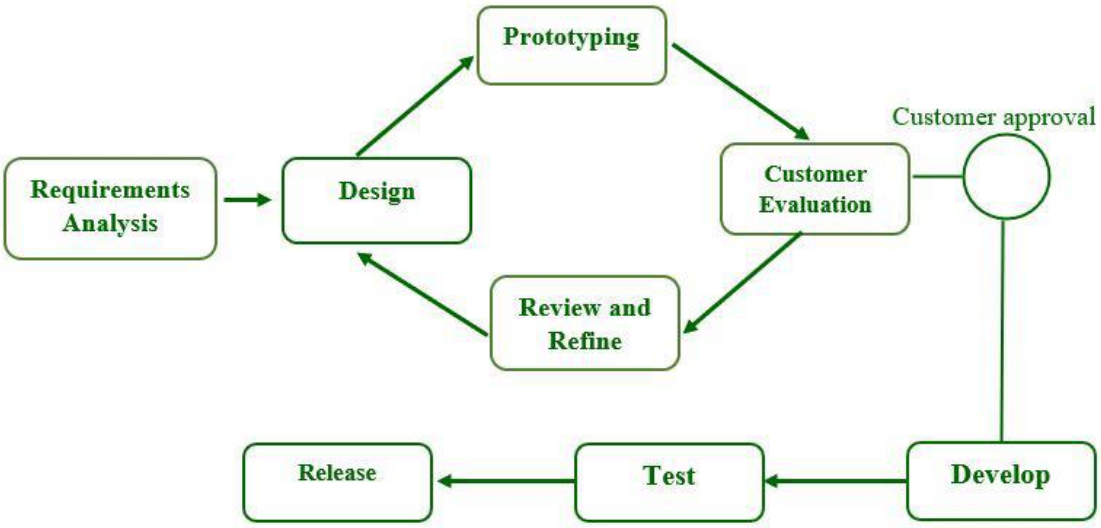
The **Prototyping Model** is a software development model in which a **working model (prototype)** of the system is built **before the actual system**.
This prototype helps users and developers **understand, clarify, and refine requirements** through continuous feedback.

Purpose of Prototyping

- To understand **unclear or incomplete requirements**
- To improve **user–developer communication**
- To reduce requirement-related risks
- To build a system that closely matches user expectations

Working Principle

- Initial requirements are gathered.
- A **quick design** is created.
- A prototype is developed and shown to the user.
- User feedback is collected.
- The prototype is refined repeatedly.
- Once requirements are finalized, the **actual system** is developed.



Phases of Prototyping Model

1. Requirement Gathering (Initial)

- Only basic and high-level requirements are identified.
- Focus is on user interface and core functionality.

2. Quick Design

- A rough design is prepared.
- Concentrates mainly on **input/output formats** and user interaction.

3. Build Prototype

- A working prototype is developed.
- It may not be fully functional or optimized.
- Speed is prioritized over quality.

4. User Evaluation

- The prototype is demonstrated to users.
- Users provide feedback on functionality, usability, and requirements.

5. Refinement of Prototype

- Changes are incorporated based on feedback.
- Steps 3 and 4 are repeated until users are satisfied.

6. Final Product Development

- Once requirements are clearly understood, the prototype is either:
  - Discarded, or
  - Improved into the final system
- Complete development, testing, and deployment are carried out.

Diagram

Requirement Gathering
↓
Quick Design
↓
Build Prototype
↓
User Evaluation
↓
Refinement
↓
Final System

Types of Prototyping Model

1. Throwaway (Rapid) Prototyping

- Prototype is developed to understand requirements.
- It is **discarded** after requirement clarification.
- Final system is developed separately.

Used when: Requirements are unclear.

2. Evolutionary Prototyping

- Prototype is **continuously improved**.
- It evolves into the final system.
- Saves redevelopment effort.

Used when: System needs frequent updates.

3. Incremental Prototyping

- System is divided into modules.
- Prototypes are developed for each module.
- Modules are integrated later.

4. Extreme Prototyping (Web Applications)

- Phase 1: Static UI pages
- Phase 2: Simulated services
- Phase 3: Full functionality

Characteristics

- User-centric development
- Iterative refinement
- Early user involvement
- Emphasis on user interface
- Flexible requirement handling

Advantages

1. **Better Requirement Understanding**
    - Reduces ambiguity in requirements.
2. **Early User Feedback**
    - Users can see and interact with the system early.
3. **Reduced Risk**
    - Errors are identified at early stages.
4. **Improved User Satisfaction**
    - Final product matches user expectations.
5. **Ease of Changes**
    - Modifications are easier and cheaper.

Disadvantages

1. **Scope Creep**
    - Users may request continuous changes.
2. **Poor Design Quality**
    - Focus on speed may compromise architecture.
3. **Increased Cost**
    - Multiple iterations increase effort.
4. **User Confusion**
    - Users may mistake prototype as final product.
5. **Not Suitable for Large Systems**
    - Difficult to manage complex systems.

When to Use Prototyping Model

- Requirements are **not well defined**
- High user interaction systems
- New or innovative applications
- User interface-intensive software

Example

ATM System

- Prototype shows screen flow (login, withdraw, balance check)
- User feedback refines functionality and usability
- Final system is developed after requirement confirmation

Topic 11: Spiral Model

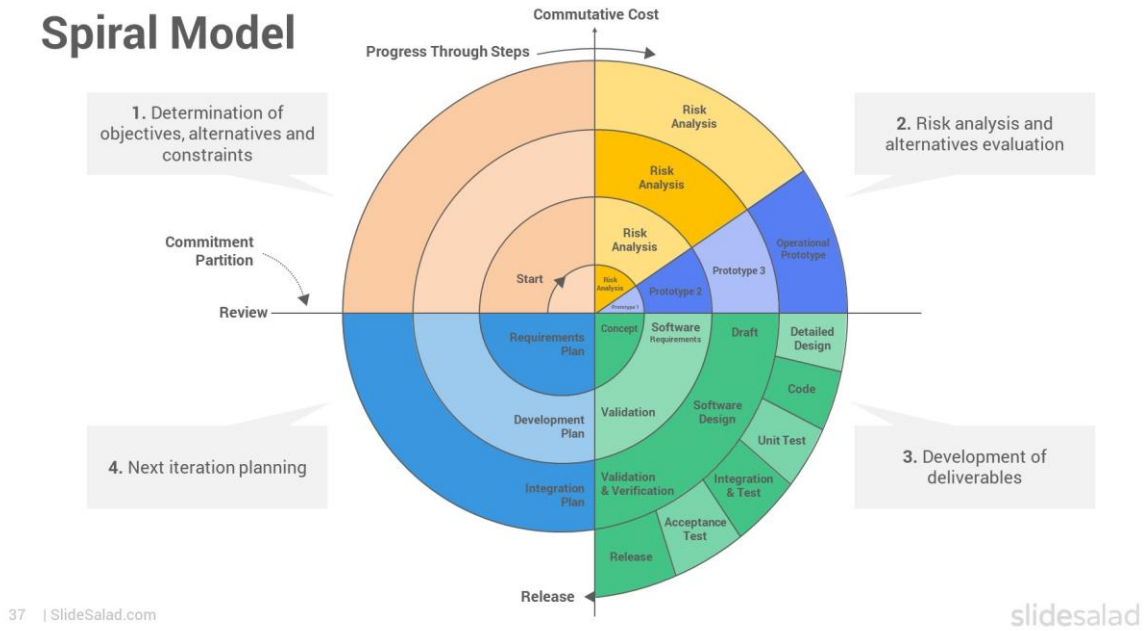
Definition

The **Spiral Model** is a **risk-driven evolutionary process model** that combines features of the **Waterfall model** and **Iterative development**.

In this model, software is developed in a series of **spiral loops**, where each loop represents a **phase of development**. The main focus of the spiral model is **risk analysis and risk reduction**.

Basic Concept

- Development proceeds in **iterations (loops of the spiral)**.
- Each loop addresses a set of objectives and risks.
- At the end of every loop, a **review is conducted** to decide whether to proceed further.
- The product **evolves gradually** with each iteration.



Phases of Spiral Model (Four Quadrants)

Each loop of the spiral consists of **four quadrants**:

1. Planning (Objective Setting)

- Identify objectives for the current phase.
- Identify alternatives and constraints.
- Define requirements for that iteration.

2. Risk Analysis

- Identify technical, managerial, and schedule risks.
- Evaluate alternatives.
- Develop prototypes or simulations to reduce risks.

3. Engineering (Development & Testing)

- Design, coding, and testing are carried out.
- The product is built for the current iteration.

4. Evaluation (Customer Review)

- Customer evaluates the developed product.

- Feedback is obtained.
- Decision is made to continue, modify, or stop the project.

Characteristics of Spiral Model

- Risk-oriented approach
- Iterative and incremental development
- Continuous customer involvement
- Strong emphasis on planning and evaluation
- Suitable for large and complex projects

Advantages

1. **Effective Risk Management**
    - Risks are identified and resolved early.
2. **Customer Feedback**
    - User evaluation after each iteration.
3. **Flexibility**
    - Changes can be incorporated easily.
4. **High Quality Software**
    - Continuous verification and validation.
5. **Early Detection of Errors**
    - Issues are found early in development.

Disadvantages

1. **Complex Management**
    - Requires expertise in risk analysis.
2. **High Cost**
    - Risk assessment and prototyping increase cost.
3. **Not Suitable for Small Projects**
    - Overhead is too high.
4. **Difficult to Schedule**
    - Number of iterations is not fixed.

When to Use Spiral Model

- Large, complex, and high-risk projects
- Systems with unclear or evolving requirements
- Projects involving new technology
- Mission-critical systems

Example

Online Banking System

- Loop 1: Requirement identification and risk analysis
- Loop 2: Security and transaction risks
- Loop 3: Performance and scalability risks
- Final loop: Complete system integration

Topic 12: RAD (Rapid Application Development) Model

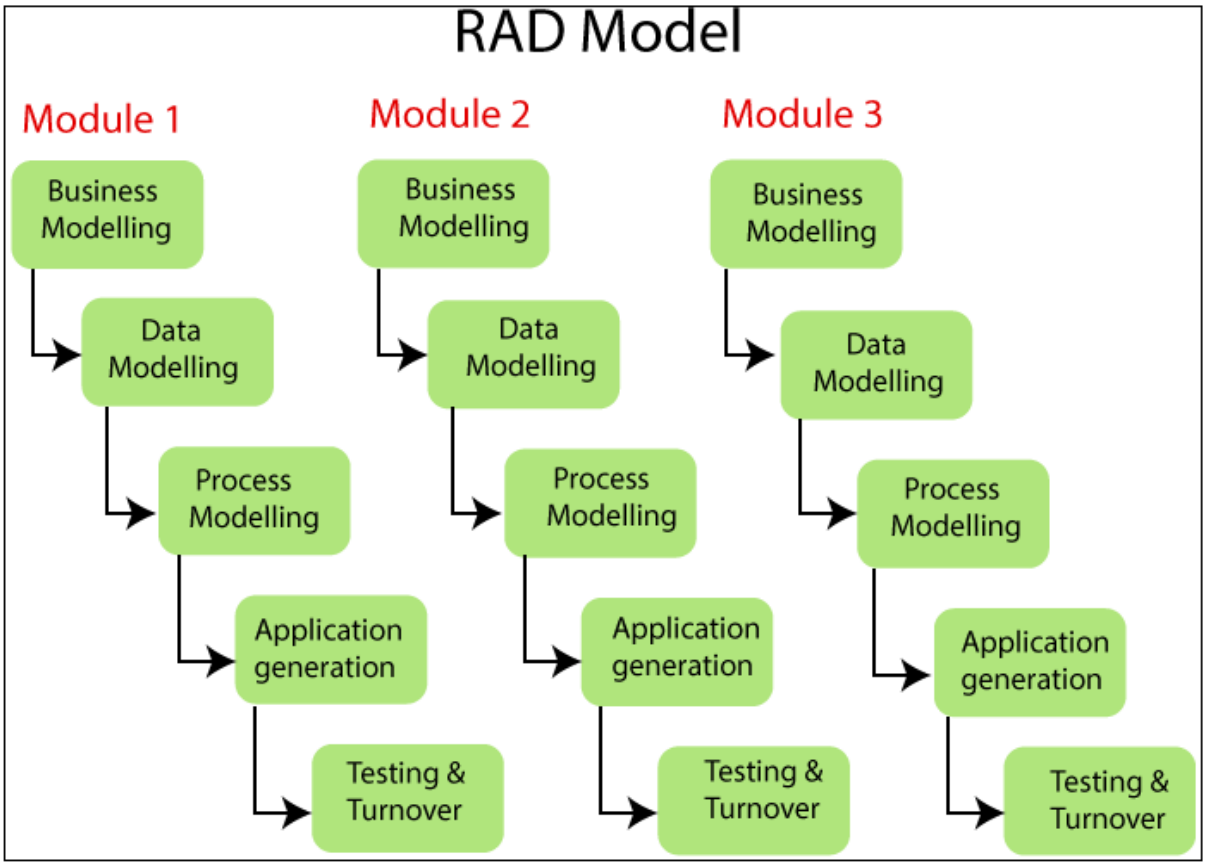
Definition

The **RAD Model** is a **software development process model** that emphasizes **rapid development and quick delivery** by using **prototyping, reusable components, and parallel development**.

The main goal of RAD is to **reduce development time** while maintaining acceptable quality through **continuous user involvement**.

Basic Idea of RAD

- Development is done in a **very short time frame** (typically 60–90 days).
- The system is divided into **small, manageable modules**.
- Each module is developed **in parallel** by different teams.
- Heavy use of **prototypes, CASE tools, and reusable components**.
- User feedback is continuous.



Phases of RAD Model

1. Business Modeling

- Identify business functions and information flow.
- Understand how data is exchanged between business processes.
- Focus is on **what the system must do**, not how.

2. Data Modeling

- Define data objects required by the business.
- Identify attributes and relationships.
- Convert business information into data models.

3. Process Modeling

- Define processes that manipulate data objects.
- Describe workflows, business rules, and operations.
- CRUD operations (Create, Read, Update, Delete) are identified.

4. Application Generation

- Actual coding is done.
- Use of:
  - Reusable software components
  - 4GLs (Fourth Generation Languages)
  - CASE tools
- Prototypes are rapidly converted into working systems.

5. Testing and Turnover

- Unit, integration, and system testing are performed.
- Since components are reused, testing time is reduced.
- System is deployed and handed over to the customer.

Characteristics of RAD Model

- Short development cycles
- Modular-based development
- Parallel development approach
- Heavy user involvement
- Use of automated tools

Advantages

1. **Very Fast Development**
   - Reduces development time significantly.
2. **Early User Feedback**
   - Continuous interaction ensures user satisfaction.
3. **Flexibility**
   - Changes can be incorporated easily.
4. **Reusability**
   - Existing components reduce effort and cost.
5. **Improved Productivity**
   - Parallel development speeds up delivery.

Disadvantages

1. **Requires Skilled Developers**
   - Team must be experienced and well-coordinated.
2. **Not Suitable for Large, Complex Systems**
   - Difficult to modularize very large projects.
3. **High Dependency on User Availability**
   - Requires continuous user participation.
4. **Costly Tools**
   - CASE tools and RAD environments may be expensive.

When to Use RAD Model

- Requirements are **well understood**
- System can be **modularized**
- Short time-to-market is critical
- User is available for frequent feedback
- Projects with **low technical risk**

When NOT to Use RAD Model

- Systems with high complexity or performance constraints
- Safety-critical or mission-critical systems

- Projects with unclear requirements
- Limited user involvement

Example
Student Information System
- Module 1: Student registration
- Module 2: Attendance management
- Module 3: Results and reports

Each module is developed quickly and integrated.

Topic 13: Agile Model – Introduction

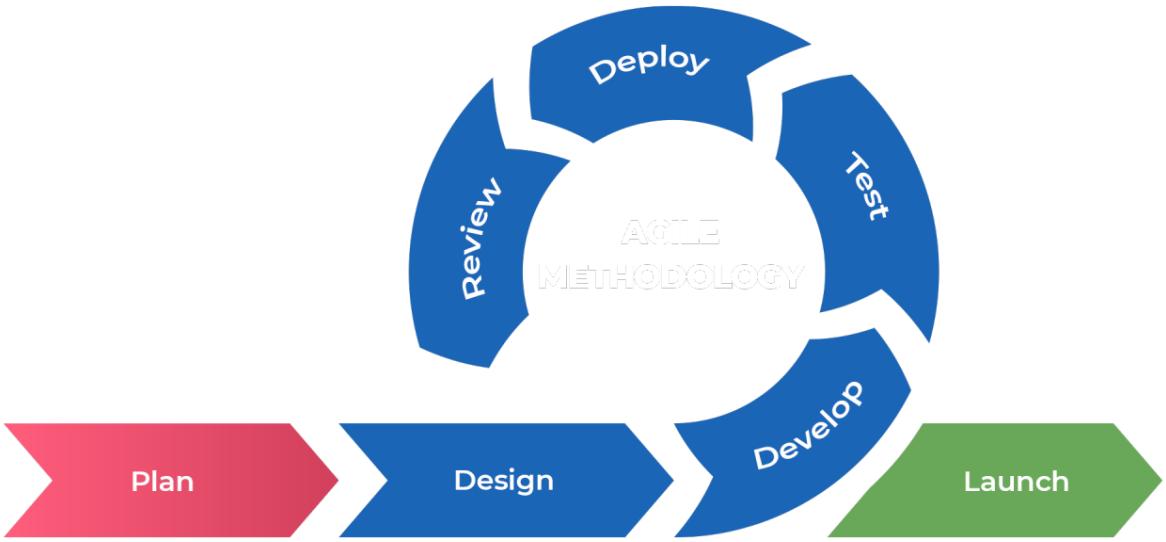
Definition
The **Agile Model** is a **modern software development approach** that focuses on **iterative development, continuous customer collaboration, and quick response to change**.
Instead of developing the entire system at once, Agile breaks the project into **small, manageable iterations (called sprints)**, each delivering a **working piece of software**.

Need for Agile Model
Traditional models like **Waterfall** assume that requirements are fixed. In real-world projects:
- Requirements change frequently
- Customer feedback is needed early
- Fast delivery is expected

The Agile model was introduced to **handle changing requirements efficiently** and deliver **high-quality software quickly**.

Basic Principles of Agile
Agile development is based on the **Agile Manifesto**, which emphasizes:
- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Key Characteristics of Agile Model
- **Iterative and incremental development**
- **Short development cycles (sprints)** – usually 1 to 4 weeks
- **Continuous customer involvement**
- **Frequent delivery of working software**
- **Adaptability to change**
- **Emphasis on teamwork and communication**



Agile Development Process (Overview)

1. **Requirement Gathering**
   - Requirements are captured as **user stories**.
2. **Sprint Planning**
   - Select user stories for the current sprint.
3. **Design & Development**
   - Design and coding are done together.
   - Small, functional increments are developed.
4. **Testing**
   - Continuous testing is performed during each sprint.
5. **Review & Feedback**
   - Working software is demonstrated to the customer.
6. **Deployment**
   - Increment is delivered and released.
7. **Next Sprint**
   - Process repeats until the system is complete.

Diagram (Conceptual)

Plan → Design → Develop → Test → Review → Release

↑_____|

(Repeated in Sprints)

Popular Agile Methodologies

- **Scrum**
- **Extreme Programming (XP)**
- **Kanban**
- **Lean Software Development**
- **Feature-Driven Development (FDD)**

Advantages of Agile Model

1. **Handles Changing Requirements**
   - Changes can be introduced even late in development.

2. **Early and Continuous Delivery**
    - Working software is available early.
3. **High Customer Satisfaction**
    - Regular feedback ensures alignment with user needs.
4. **Improved Quality**
    - Continuous testing and integration.
5. **Reduced Risk**
    - Problems are identified early.

Disadvantages of Agile Model

1. **Less Documentation**
    - Can be an issue for maintenance or compliance.
2. **Requires Strong Customer Involvement**
    - Lack of user availability affects progress.
3. **Difficult to Predict Cost and Schedule**
    - Scope changes frequently.
4. **Not Suitable for All Projects**
    - Less effective for very large or safety-critical systems.

When to Use Agile Model

- Requirements are **frequently changing**
- Customer is available for collaboration
- Fast time-to-market is required
- Small to medium-sized projects
- Dynamic business environments

Topic 14: Introduction to Extreme Programming (XP)

Definition

Extreme Programming (XP) is an **Agile software development methodology** that focuses on **high-quality software, customer satisfaction, and rapid response to changing requirements**.

Why Extreme Programming?

Traditional development models struggle when:

- Requirements change frequently
- Customers want early delivery
- Quality issues arise late

XP was introduced to:

- Improve **software quality**
- Reduce **development risk**
- Encourage **continuous customer involvement**
- Support **frequent requirement changes**

Core Values of XP

XP is based on **five core values**:

1. **Communication**
    - Continuous interaction among team members and customers.
2. **Simplicity**
    - Design only what is needed today.
3. **Feedback**

- Frequent feedback from tests and customers.

4. **Courage**
   - Courage to refactor, change design, and accept feedback.
5. **Respect**
   - Mutual respect within the development team.

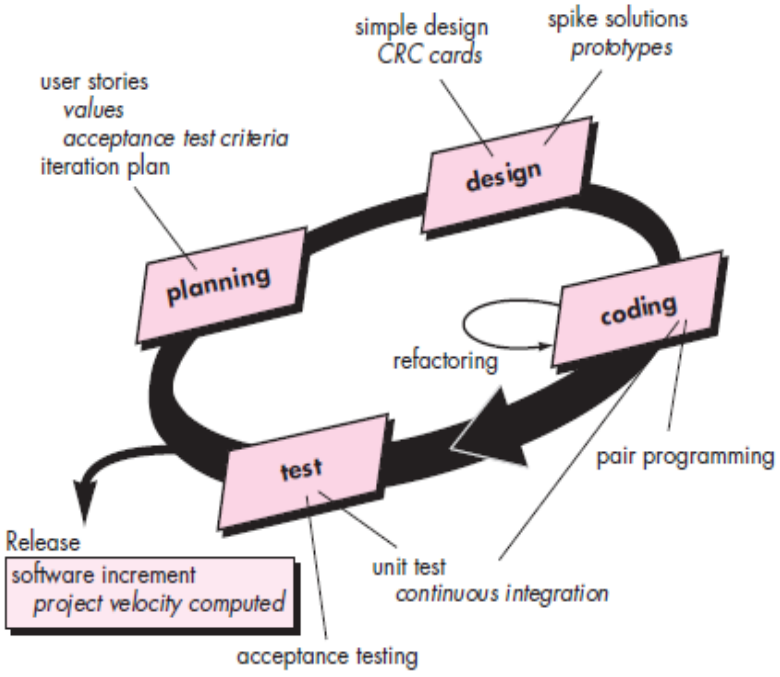
XP Process (Overview)

The XP development life cycle consists of the following phases:

1. **Planning**
   - Requirements are collected as **user stories**.
   - Release planning is done.
2. **Design**
   - Simple design principles are followed.
   - Use of **CRC cards** (Class–Responsibility–Collaboration).
3. **Coding**
   - Code is written using **pair programming**.
   - Coding standards are strictly followed.
4. **Testing**
   - **Test-Driven Development (TDD)** is used.
   - Unit tests and acceptance tests are mandatory.
5. **Release**
   - Small and frequent releases are delivered.

Diagram (Conceptual)

FIGURE 3.2
The Extreme Programming process



Key Practices of Extreme Programming

1. **Pair Programming**
   - Two programmers work at one workstation.
2. **Test-Driven Development (TDD)**
   - Write tests before writing code.
3. **Continuous Integration**
   - Code is integrated and tested frequently.

4. **Refactoring**
    - Improve code structure without changing functionality.
5. **Simple Design**
    - Avoid unnecessary complexity.
6. **Small Releases**
    - Deliver working software frequently.
7. **On-Site Customer**
    - Customer is available for continuous feedback.
8. **Coding Standards**
    - Uniform coding style across the team.

Characteristics of XP

- Short development cycles
- High customer involvement
- Strong emphasis on testing
- Frequent feedback
- Adaptability to change

Advantages of XP

1. **High Software Quality**
    - Continuous testing and refactoring.
2. **Customer Satisfaction**
    - Frequent feedback ensures correct product.
3. **Flexibility**
    - Easy to incorporate changing requirements.
4. **Early Delivery**
    - Small, frequent releases.
5. **Improved Team Collaboration**
    - Pair programming and communication.

Disadvantages of XP

1. **Requires High Discipline**
    - Strict practices must be followed.
2. **Not Suitable for Large Teams**
    - Best for small to medium projects.
3. **Customer Availability Needed**
    - On-site customer is essential.
4. **Less Documentation**
    - May cause issues in maintenance.

When to Use Extreme Programming

- Requirements change frequently
- Small to medium-sized projects
- Skilled and collaborative team
- Customer is readily available
- Quality is a top priority

Topic 15: Introduction to Scrum

Definition

Scrum is an **Agile framework** used for developing, delivering, and maintaining **complex software systems**.

It focuses on **iterative development**, **team collaboration**, and **frequent delivery of working software** through short, time-boxed iterations called **Sprints**.

Purpose of Scrum

Scrum was introduced to:

- Handle **frequently changing requirements**
- Improve **team productivity**
- Deliver **high-quality software quickly**
- Ensure **continuous customer feedback**
- Increase transparency and adaptability

Basic Concept of Scrum

- Development is done in **small cycles called Sprints** (usually 1–4 weeks).
- Each Sprint results in a **potentially shippable product increment**.
- Requirements are maintained in a **Product Backlog**.
- Scrum promotes **self-organizing, cross-functional teams**.

Scrum Framework Overview

Scrum is built around **three main components**:

1. **Roles**
2. **Artifacts**
3. **Events (Ceremonies)**

Scrum Roles

1. Product Owner

- Represents customer and business needs
- Maintains and prioritizes the **Product Backlog**
- Defines acceptance criteria

2. Scrum Master

- Facilitates Scrum process
- Removes obstacles (impediments)
- Ensures Scrum principles are followed

3. Development Team

- Cross-functional professionals
- Responsible for designing, coding, testing, and delivering the product increment

Scrum Artifacts

1. Product Backlog

- Ordered list of all requirements
- Contains user stories, features, and bug fixes

2. Sprint Backlog

- Subset of Product Backlog selected for a Sprint
- Includes tasks needed to complete the Sprint

3. Increment

- Working product produced at the end of each Sprint
- Must meet the **Definition of Done**

Scrum Events

1. Sprint

- Time-boxed iteration (1–4 weeks)
- Core development cycle

2. Sprint Planning

- Decide **what** to build and **how** to build it

3. Daily Scrum

- 15-minute daily meeting
- Team synchronizes work and identifies blockers

4. Sprint Review

- Demonstration of completed work to stakeholders

5. Sprint Retrospective

- Team reflects on the Sprint
- Identifies improvements for the next Sprint